

Week16_CH09.추천 시스템

01 추천 시스템의 개요와 배경

추천 시스템의 유형

- 콘텐츠 기반 필터링(Content based filtering)
- 협업 필터링(Collaborative filtering)
 - 최근접 이웃(Nearest Neighbor) 협업 필터링
 - 잠재 요인(Latent Factor) 협업 필터링

02 콘텐츠 기반 필터링 추천 시스템

콘텐츠 기반 필터링

- 사용자가 특정 아이템을 매우 선호
 - → 아이템과 비슷한 콘텐츠의 다른 아이템 추천
 - 영화의 경우 장르, 감독, 출연 배우, 키워드 등을 감안



장르: SF, 드라마, 미스터리
감독: 드니 빌뇌브
출연: 에이미 아담스, 제레미 러너
키워드: 외계인 침공, 예술성, 스릴러 요소



장르: SF, 액션, 스릴러
감독: 드니 빌뇌브
출연: 라이언 고슬링, 해리슨 포드
키워드: 리들리 스콧 감독의 전작을 리메이크



장르: SF, 액션, 스릴러
감독: 리들리 스콧
출연: 노미 라마스, 마이클 패스벤더
키워드: 에일리언 프리퀼, 액션과 스릴러의 조화



사용자 선호 프로필

선호 장르: SF, 액션, 스릴러
선호 배우: 에이미 아담스, 마이클 패스벤더 등
선호 감독: 리들리 스콧, 드니 빌뇌브

03 최근접 이웃 협업 필터링

협업 필터링(Collaborative Filtering)

- 친구들에게 묻는 것과 유사
- 사용자 행동 양식 (User Behavior)만으로 추천 수행
 - 사용자기 아이템에 매긴 평점 정보
 - 상품 구매 이력 등
- 사용자 행동 데이터 기반 평가 하지 않은 아이템 예측 평가

사용자가 평가하지 않은 아이템을 평가한
아이템에 기반하여 예측 평가하는 알고리즘

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	✓
User 2	4	2		3
User 3		1	2	2

- 사용자-아이템 평점 매트릭스
 - 사용자가 평가한 다른 아이템 기반으로 예측 평가 도출
-
- 행은 개별 사용자
 - 열은 개별 아이템
 - 값은 평점

로우 레벨 형태의 사용자 - 아이템 평점 데이터

User ID	Item ID	Rating
User 1	Item 1	3
User 1	Item 3	3
User 2	Item 1	4
User 2	Item 2	1
User 3	Item 4	5

변환

사용자 로우, 아이템 칼럼으로 구성된 사용자 - 아이템 평점 데이터

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	
User 2	4	1		
User 3				5

- 이런 형태라면 판다스의 `pivot_table()` 등 함수 이용해서 변경
- 희소 행렬(Sparse Matrix) 특성 가짐
 - 아이템 평점 매기는 경우 많지 않기 때문

최근접 이웃 협업 필터링

- = 메모리 (memory) 협업 필터링
- 사용자 기반(User-User)
 - 사용자와 비슷한 고객
- 아이템 기반(Item-Item)
 - 이 상품을 선택한 다른 고객

사용자 기반 최근접 이웃 방식

- 특정 사용자와 유사한 사용자를 Top-N으로 선정
 - 사용자 간 similarity 계산
- Top-N 사용자가 좋아하는 아이템 추천

		다크 나이트	인터스텔라	엣지 오브 투모로우	프로메테우스	스타워즈 라스트제다이
상호간 유사도 높음	사용자 A	5	4	4		
	사용자 B	5	3	4	5	3
	사용자 C	4	3	3	2	5

사용자 A는 사용자 C보다 사용자 B와 영화 평점 측면에서 유사도가 높음. 따라서 사용자 A에게는 사용자 B가 재미있게 본 '프로메테우스'를 추천

아이템 기반 최근접 이웃 방식

- 아이템 간의 속성 유사도가 아님
- 아이템에 대한 사용자들의 호/불호의 평가 척도

		사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
상호간 유사도 높음	다크 나이트	5	4	5	5	5
	프로메테우스	5	4	4		5
	스타워즈 라스트제다이	4	3	3		4

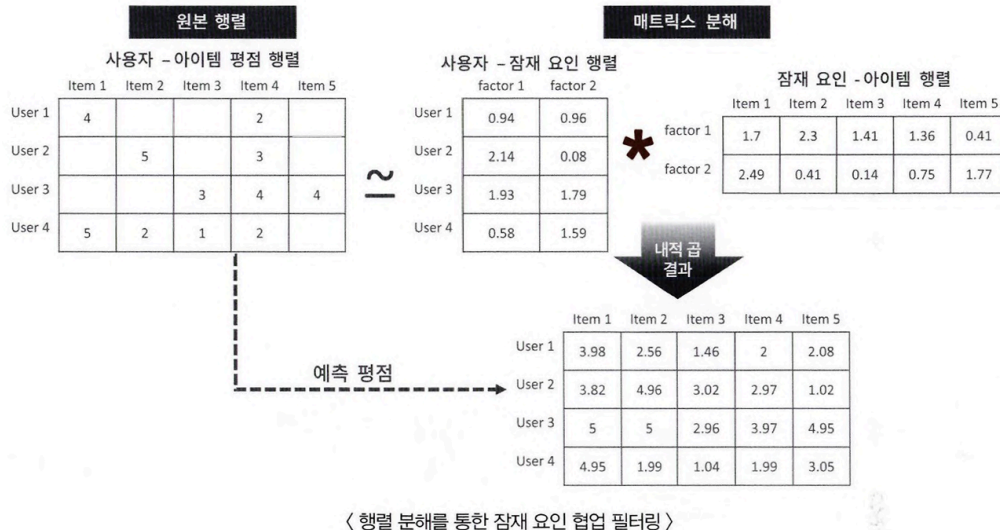
여러 사용자들의 평점을 기준으로 볼 때 '다크 나이트'와 가장 유사한 영화는 '프로메테우스'

- 일반적으로 사용자 기반보다 아이템 기반 협업 필터링이 정확도 높음
 - 유명 영화는 취향 관계 없이 관람함
 - 사용자들이 평점 매긴 영화 많지 않음

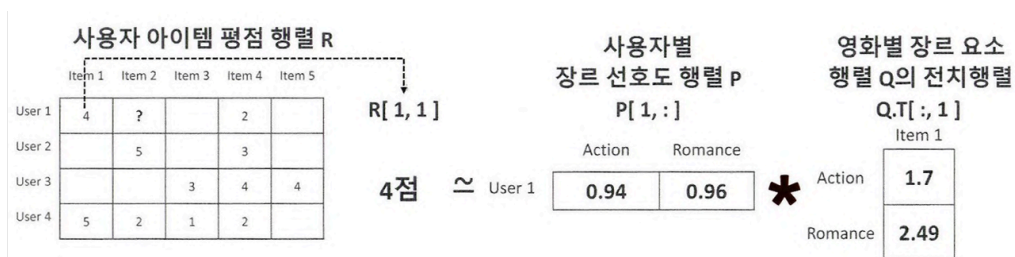
04 잠재 요인 협업 필터링

잠재 요인 협업 필터링

- 사용자-아이템 평점 매트릭스 속 잠재 요인 추출
 - 추천 예측
- 행렬 분해(Matrix Factorization)
 - 대규모 다차원 행렬을 차원 축소

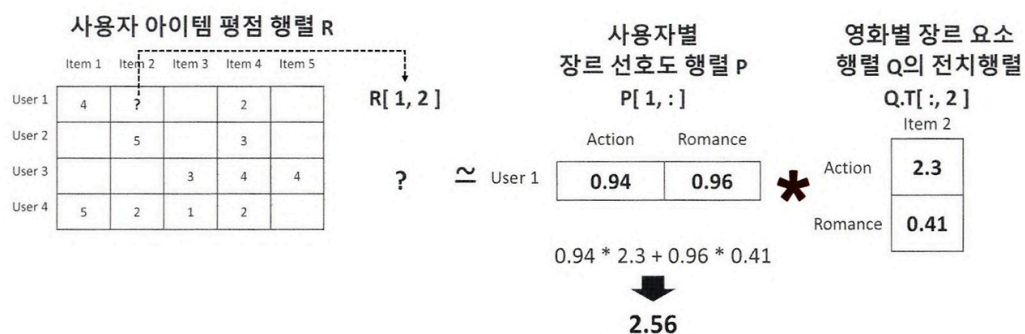


- 잠재 요인
 - 영화가 가지는 장르별 특성 선호도 등
- 사용자의 아이템에 대한 평점 $R(u, i)$
- $R(1, 1) \rightarrow 4$ 점
- $R(1, 4) \rightarrow 2$ 점
- 사용자-잠재 요인 행렬을 사용자의 영화 장르별 선호도 P 로 가정
 - factor 1: Action 선호도
 - factor 2: Romance 선호도
- $P(1, 1) \rightarrow 0.94$
- $P(1, 2) \rightarrow 0.96$



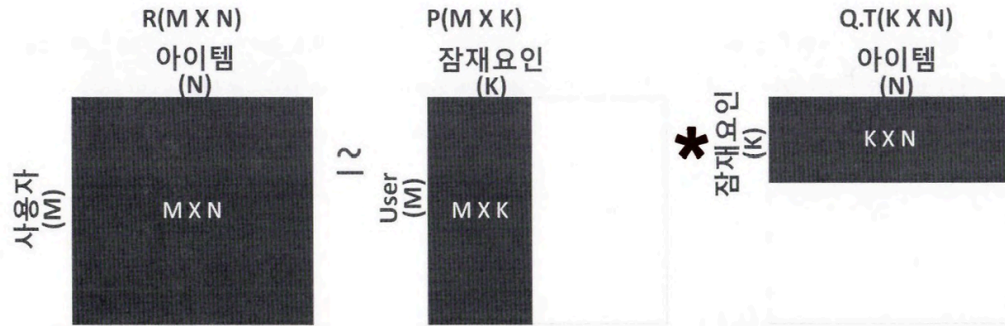
- k 는 잠재 요인 칼럼
- Q 는 P 와의 내적 계산 통해 예측 평정 계산 위해 Q 의 행과 열 위치 서로 교환
 - Q, T 로 변환

- $Q(1, 1) \rightarrow 1.7$
- $Q(2, 1) \rightarrow 2.49$
- 평점은 특정 영화 장르 선호도를 반영해 결정됨
 - 사용자의 장르별 선호도 벡터와 영화의 장르별 특성 벡터를 서로 곱해서 만들 수 있음
 - User1의 item1의 평점인 $R(1, 1)$ 의 4점
 - P 매트릭스의 User1과 Q.T 매트릭스 Item 2의 내적 곱값
 - 2.56



행렬 분해의 이해

- SVD(Singular Vector Decomposition)
- NMF(Non-Negative Matix Factorization)
 - Factorization = 인수분해
- M개의 사용자 행과 N개의 아이템 열을 가진 평점 행렬 R은 MxN 차원
 - 행렬 분해 통해서 사용자-K 차원 잠재 요인 행렬 P
 - K차원 잠재 요인 아이템 행렬 Q.T로 분해



- $R = P * Q.T$

- M은 총 사용자 수
- N은 총 아이템 수
- K는 잠재 요인의 차원 수
- R은 $M \times N$ 차원의 사용자-아이템 평점 행렬
 - P는 사용자와 잠재 요인과의 관계 값을 가지는 $M \times K$ 차원의 사용자-잠재 요인 행렬
 - Q는 아이템과 잠재 요인과의 관계 값을 가지는 $N \times K$ 차원의 아이템-잠재 요인 행렬
 - Q.T는 Q 매트릭스의 행과 열 값을 교환한 전치 행렬

- 행렬 내에 널 값 많이 가지는 고차원 희소 행렬인 R행렬은 자차원의 밀집 행렬인 Q와 P로 분해될 수 있음

R(4 X 5)

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	4			2	
User 2		5		3	
User 3			3	4	4
User 4	5	2	1	2	

P(4 X 2)

	factor 1	factor 2
User 1	0.94	0.96
User 2	2.14	0.08
User 3	1.93	1.79
User 4	0.58	1.59

Q.T(2 X 4)

	Item 1	Item 2	Item 3	Item 4	Item 5
factor 1	1.7	2.3	1.41	1.36	0.41
factor 2	2.49	0.41	0.14	0.75	1.77

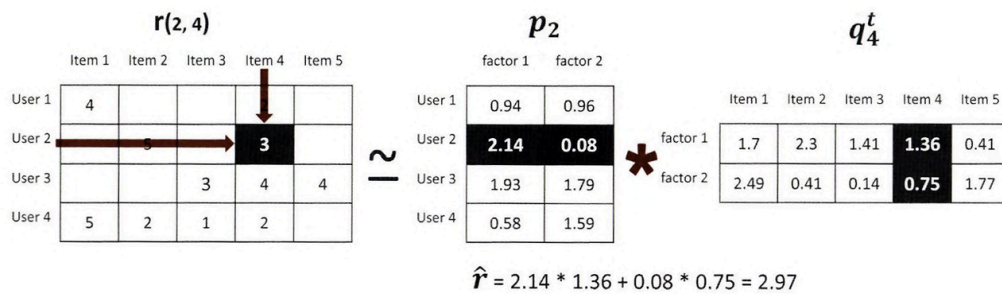
개별 Column은 개별 item에 대한 Latent Factor들을 반영

개별 Row는 개별 User에 대한 Latent Factor들을 반영

- R행렬 u행 사용자와 i열 아이템 위치에 있는 평점 데이터 $r(u,i)$

$$r_{(u,i)} = p_u * q_i^t$$

- R행렬 2행 사용자와 4열 아이템에 해당하는 $r(2, 4)$ 평점 3점은
 - $r(2, 4) = p_2 * q_4^t$
 - $r(2, 4) = 2.14 * 1.36 + 0.08 * 0.75 = 2.97$



- 사용자가 평가하지 않은 아이템에 대한 평점도 잠재 요인으로 분해된 P, Q 이용해 예측 가능
- 사용자-아이템 평점 행렬의 미정 값 포함 모든 평점 값은 행렬 분해 통해 얻어진 P, Q.T 행렬 내적 통해 다시 계산 가능
- 널값이 있는 행렬에는 확률적 경사 하강법 방식 이용

확률적 경사 하강법을 이용한 행렬 분해

SGD 기반 행렬 분해

- P와 Q 행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록
 - 반복적인 비용 함수 최적화를 통해 P와 Q를 유추해내는 것
- SGD 이용한 행렬 분해 절차
 1. P와 Q를 임의의 값을 가진 행렬로 설정
 2. P와 Q.T 값을 곱해 예측 R 행렬과 실제 R 행렬에 해당하는 오류 값을 계산
 3. 오류값을 최소화할 수 있도록 P와 Q 행렬을 적절한 값으로 각각 업데이트
 4. 만족할 만한 오류 값 가질 때 까지 2-3 작업 반복, P와 Q 값을 업데이트하여 근사화
- 실제 값과 예측값의 오류 최소화와 L2 Regularization 고려한 비용 함수 식

$$\min \sum (r_{(u,i)} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

- 비용 함수 최소화하기 위해 새롭게 업데이트 되는 식

$$\dot{p}_u = p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u)$$

$$\dot{q}_i = q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i)$$

- p_u : P 행렬의 사용자 u행 벡터
 - q_i^t : Q 행렬의 아이템 i행의 전치 벡터(transpose vector)
 - $r_{(u,i)}$: 실제 R 행렬의 u행, i열에 위치한 값.
 - $\hat{r}_{(u,i)}$: 예측 R 행렬의 u행, i열에 위치한 값. $p_u * q_i^t$ 로 계산
 - $e_{(u,i)}$: u행, i열에 위치한 실제 행렬 값과 예측 행렬 값의 차이 오류. $r_{(u,i)} - \hat{r}_{(u,i)}$ 로 계산
 - η : SGD 학습률
 - λ : L2 규제(Regularization) 계수
- L2 규제를 반영해 실제 R 행렬 값과 예측 R 행렬 값의 차이를 최소화하는 방향성 가짐
 - P, Q 행렬에 업데이트 값 반복적 수행
 - 최적화된 예측 R 행렬을 구하는 방식

- 원본 행렬 R을 미정인 널 값을 포함해 생성
- 분해 행렬 P, Q는 정규 분포를 가진 랜덤 값으로 초기화
- 잠재 요인 차원은 3

```

import numpy as np

R = np.array([[4, np.nan, np.nan, 2, np.nan],
              [np.nan, 5, np.nan, 3, 1],
              [np.nan, np.nan, 3, 4, 4],
              [5, 2, 1, 2, np.nan]])

num_users, num_items = R.shape
K=3

np.random.seed(1)
P = np.random.normal(scale=1./K, size=(num_users, K))
Q = np.random.normal(scale=1./K, size=(num_items, K))

```

- 실제 R 행렬과 예측 행렬의 오차를 구하는 get_rmse() 함수 생성

```

from sklearn.metrics import mean_squared_error

def get_rmse(R, P, Q, non_zeros):
    error = 0

    full_pred_matrix = np.dot(P, Q.T)

    x_non_zero_ind = [non_zero[0] for non_zero in non_zeros]
    y_non_zero_ind = [non_zero[1] for non_zero in non_zeros]
    R_non_zeros = R[x_non_zero_ind, y_non_zero_ind]
    full_pred_matrix_non_zeros = full_pred_matrix[x_non_zero_ind, y_non_zero_ind]

    mse = mean_squared_error(R_non_zeros, full_pred_matrix_non_zeros)
    rmse = np.sqrt(mse)

    return rmse

```

- SGD 기반 행렬 분해 수행
- R에서 널 값 제외한 데이터의 행렬 인덱스 추출
- steps: SGD 반복 업데이트 횟수
- 새로운 pu, qi 값으로 업데이트

$$\dot{p}_u = p_u + \eta(e_{(u,i)} * q_i - \lambda * p_u), \dot{q}_i = q_i + \eta(e_{(u,i)} * p_u - \lambda * q_i)$$

```

non_zeros = [(i, j, R[i, j]) for i in range(num_users) for j in range(num_items) if R[i, j]]

steps = 1000
learning_rate = 0.01
r_lambda = 0.01

for step in range(steps):
    for i, j, r in non_zeros:
        eij = r - np.dot(P[i, :], Q[j, :].T)
        P[i, :] = P[i, :] + learning_rate * (eij * Q[j, :] - r_lambda * P[i, :])
        Q[j, :] = Q[j, :] + learning_rate * (eij * P[i, :] - r_lambda * Q[j, :])

    rmse = get_rmse(R, P, Q, non_zeros)
    if (step % 50) == 0:
        print("### iteration step : ", step, " rmse : ", rmse)

## iteration step : 0 rmse : 3.261355059488935
## iteration step : 0 rmse : 3.26040057174686
## iteration step : 0 rmse : 3.253984404542389
## iteration step : 0 rmse : 3.2521583839863624
## iteration step : 0 rmse : 3.252335303789125
## iteration step : 0 rmse : 3.251072196430487
## iteration step : 0 rmse : 3.2492449982564864
## iteration step : 0 rmse : 3.247416477570409
## iteration step : 0 rmse : 3.241926055455223
## iteration step : 0 rmse : 3.2400454107613084
## iteration step : 0 rmse : 3.240166740749792
## iteration step : 0 rmse : 3.2388050277987723
## iteration step : 50 rmse : 0.5003190892212748
## iteration step : 50 rmse : 0.5001616291326989
## iteration step : 50 rmse : 0.49899601202578087
## iteration step : 50 rmse : 0.4988483450145831
## iteration step : 50 rmse : 0.49895189256631756
## iteration step : 50 rmse : 0.49833236830090993
## iteration step : 50 rmse : 0.4984149498979791

```

- 분해된 P와 Q 함수를 P*Q.T로 예측 행렬 만들어서 출력

```

pred_matrix = np.dot(P, Q.T)
print('예측 행렬 : \n', np.round(pred_matrix, 3))

## 예측 행렬 :
## [[3.991 0.897 1.306 2.002 1.663]
## [6.696 4.978 0.979 2.981 1.003]
## [6.677 0.391 2.987 3.977 3.986]
## [4.968 2.005 1.006 2.017 1.14 ]]

```


08 파이썬 추천 시스템 패키지 - Surprise

Surprise를 이용한 추천 시스템 구축

- Surprise의 관련 모듈 임포트

```
!pip install numpy==1.24.4
!pip install scikit-surprise
```

숨겨진 출력 표시

```
import surprise
print(surprise.__version__)
```

1.1.4

```
from surprise import SVD
from surprise import Dataset
from surprise import accuracy
from surprise.model_selection import train_test_split
```

```
data = Dataset.load_builtin('ml-100k')
```

- 추천 위한 데이터 세트 로딩
- Surprise 데이터 로딩은 Dataset 클래스를 이용해서만 가능
- 주요 데이터가 row 레벨 형태로 돼 있는 포맷 데이터만 처리
- 무비렌즈 과거 버전 데이터 가져오는 API 제공
- load_builtin() 함수 사용

```
data = Dataset.load_builtin('ml-100k')

trainset, testset = train_test_split(data, test_size=.25, random_state=0)

Dataset ml-100k could not be found. Do you want to download it? [Y/n] Y
Trying to download dataset from https://files.grouplens.org/datasets/movielens/ml-100k.zip
Done! Dataset ml-100k has been saved to /root/.surprise_data/ml-100k
```

- SVD로 잠재 요인 협업 필터링 수행
- 적용하는 데이터 세트는 앞에서 소개한 train_test_split()으로 분리된 학습 데이터 세트
- algo = SVD() 와 같이 알고리즘 객체를 생성
- 추천 예측 메서드는 test(), predict()
- test()
 - 사용자 - 아이템 평점 데이터 세트 전체에 대해 추천 예측
 - 입력된 데이터 세트에 대해 추천 데이터 세트 만들어줌
- predict()
 - 개별 사용자와 영화에 대한 추천 평점 반환

```
algo = SVD(random_state=0)
algo.fit(trainset)

<surprise.prediction_algorithms.matrix_factorization.SVD at 0x7affc8fefa10>

predictions = algo.test( testset )
print('prediction type :', type(predictions), ' size:', len(predictions))
print('prediction 결과의 최초 5개 추출')
predictions[:5]

prediction type : <class 'list'> size: 25000
prediction 결과의 최초 5개 추출
[Prediction(uid='120', iid='282', r_ui=4.0, est=3.5114147666251547, details=
{'was_impossible': False}),
 Prediction(uid='882', iid='291', r_ui=4.0, est=3.573872419581491, details={'was_impossible':
False}),
 Prediction(uid='535', iid='507', r_ui=5.0, est=4.033583485472447, details={'was_impossible':
False}),
 Prediction(uid='697', iid='244', r_ui=5.0, est=3.8463639495936905, details=
{'was_impossible': False}),
 Prediction(uid='751', iid='385', r_ui=4.0, est=3.1807542478219157, details=
{'was_impossible': False})]
```

- was_impossible 값이 True이면 예측값 생성할 수 없는 데이터라는 뜻

- Prediction 객체의 uid, iid, r_ui, est 등의 속성에 접근하려면 객체명.uid 형식
- 3개의 prediction 객체에서 uid, iid, est 속성 추출
- predict() 통해 추천 예측
- 인자로 개별 사용자 아이디, 아이템 아이디 입력
 - 추천 예측 평점을 포함한 정보 반환

```
[ (pred.uid, pred.iid, pred.est) for pred in predictions[:3] ]

[('120', '282', 3.5114147666251547),
 ('882', '291', 3.573872419581491),
 ('535', '507', 4.033583485472447)]

uid = str(196)
iid = str(302)
pred = algo.predict(uid, iid)
print(pred)

user: 196      item: 302      r_ui = None    est = 4.49    {'was_impossible': False}
```

- 추천 예측 평점을 est로 반환
- 테스트 데이터 세트 이용해 추천 예측 평점-실제 평점 차이 평가
- accuracy 모듈은 RMSE, MSE 등의 방법으로 성능 평가 정보 제공

```
accuracy.rmse(predictions)

RMSE: 0.9467
0.9466860806937948
```

Surprise 주요 모듈 소개

API 명	내용
<code>Dataset.load_builtin</code> (name='ml-100k')	무비렌즈 아카이브 FTP 서버에서 무비렌즈 데이터를 내려받습니다. ml-100k, ml-1M를 내려받을 수 있습니다. 일단 내려받은 데이터는 .surprise_data 디렉터리 밑에 저장되고, 해당 디렉터리에 데이터가 있으면 FTP에서 내려받지 않고 해당 데이터를 이용합니다. 입력 파라미터인 name으로 대상 데이터가 ml-100k인지 ml-1m인지를 입력합니다(name='ml-100k'). 디폴트는 ml-100k입니다.
<code>Dataset.load_from_file</code> (file_path, reader)	OS 파일에서 데이터를 로딩할 때 사용합니다. coma, 탭 등으로 칼럼이 분리된 포맷의 OS 파일에서 데이터를 로딩합니다. 입력 파라미터로 OS 파일명, Reader로 파일의 포맷을 지정합니다.
<code>Dataset.load_from_df</code> (df, reader)	판다스의 DataFrame에서 데이터를 로딩합니다. 파라미터로 DataFrame을 입력받으며 DataFrame 역시 반드시 3개의 칼럼인 사용자 아이디, 아이템 아이디, 평점 순으로 칼럼 순서가 정해져 있어야 합니다. 입력 파라미터로 DataFrame 객체, Reader로 파일의 포맷을 지정합니다.

OS 파일 데이터를 Surprise 데이터 세트로 로딩

- ratings.csv 파일은 맨 처음 위치에 칼럼명을 헤더로 가짐
- 판다스 DataFrame의 to_csv() 함수 이용해 칼럼 헤더 삭제
- 새로운 파일은 ratings_noh.csv로 저장

```
import pandas as pd

ratings = pd.read_csv('ratings.csv')

ratings.to_csv('ratings_noh.csv', index=False, header=False)
```

- ratings_noh.csv를 Dataset 모듈의 load_from_file() 이용해 Dataset로 로드
- reader 클래스 이용해 데이터 파일의 파싱 포맷 정의
- reader 클래스는 로딩될 ratings_noh.csv 파일의 파싱 정보 알려줌
- ratings_noh.csv는 칼럼 헤더 없음, 4개의 칼럼이 콤마로 분리됨
- load_from_file()로 생성된 reader 객체를 참조해 데이터 파일 파싱하며 로딩

```
ratings.to_csv('ratings_noh.csv', index=False, header=False)

from surprise import Reader

reader = Reader(line_format='user item rating timestamp', sep=',', rating_scale=(0.5, 5))
data=Dataset.load_from_file('ratings_noh.csv', reader=reader)
```

- Reader 클래스의 주요 생성 파라미터

- `line_format` (string): 칼럼을 순서대로 나열합니다. 입력된 문자열을 공백으로 분리해 칼럼으로 인식합니다.
- `sep` (char): 칼럼을 분리하는 분리자이며, 디폴트는 '\t'입니다. 판다스 DataFrame에서 입력받을 경우에는 기재할 필요가 없습니다.
- `rating_scale` (tuple, optional): 평점 값의 최소 ~ 최대 평점을 설정합니다. 디폴트는 (1, 5)이지만 ratings.csv 파일의 경우는 최소 평점이 0.5, 최대 평점이 5이므로 (0.5, 5)로 설정했습니다.

- SVD 행렬 분해 기법 이용해 추천 예측
- 잠재 요인 크기 K값
 - 파라미터 `n_factors=50`
- RMSE로 평가

```
trainset, testset = train_test_split(data, test_size=.25, random_state=0)

algo = SVD(n_factors=50, random_state=0)

algo.fit(trainset)
predictions = algo.test( testset )
accuracy.rmse(predictions)

RMSE: 0.8682
0.8681952927143516
```

판다스 DataFrame에서 Surprise 데이터 세트로 로딩

- DataFrame 역시 userid, itemid, 평점 칼럼 순서 지켜야 함
- Dataset.load_from_df(ratings[['userid', 'movieid', 'rating']], reader) 입력하면 됨

```
import pandas as pd
from surprise import Reader, Dataset

ratings = pd.read_csv('ratings.csv')
reader = Reader(rating_scale=(0.5, 5.0))

data = Dataset.load_from_df(ratings[['userid', 'movieid', 'rating']], reader)
trainset, testset = train_test_split(data, test_size=.25, random_state=0)

algo = SVD(n_factors=50, random_state=0)
algo.fit(trainset)
predictions = algo.test(testset)
accuracy.rmse(predictions)
```

RMSE: 0.8682
0.8681952927143516

Surprise 추천 알고리즘 클래스

클래스명	설명
SVD	행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘.
KNNBasic	최근접 이웃 협업 필터링을 위한 KNN 알고리즘.
BaselineOnly	사용자 Bias와 아이템 Bias를 감안한 SGD 베이스라인 알고리즘.

- SVD 클래스의 입력 파라미터

파라미터명	내용
n_factors	잠재 요인 K의 개수. 디폴트는 100, 커질수록 정확도가 높아질 수 있으나 과적합 문제가 발생할 수 있습니다.
n_epochs	SGD(Stochastic Gradient Descent) 수행 시 반복 횟수, 디폴트는 20.
biased (bool)	베이스라인 사용자 편향 적용 여부이며, 디폴트는 True입니다.

- 알고리즘 유형

알고리즘 유형	RMSE	MAE	Time
SVD	0.934	0.737	0:00:11
SVD++	0.92	0.722	0:09:03
NMF	0.963	0.758	0:00:15
Slope One	0.946	0.743	0:00:08
k-NN	0.98	0.774	0:00:10
Centered k-NN	0.951	0.749	0:00:10
k-NN Baseline	0.931	0.733	0:00:12
Co-Clustering	0.963	0.753	0:00:03
Baseline	0.944	0.748	0:00:01

베이스라인 평점

- 개인의 성향을 반영해 아이템 평가에 편향성(bias) 요소를 반영하여 평점 부과하는 것
- 전체 평균 평점 + 사용자 편향 점수 + 아이템 편향 점수 공식으로 계산 됨


 모든 사용자의 평균 영화 평점 : **3.5**


 난 진정한 영화 매니아.
영화 평가는 언제나 깐깐하게
 사용자 A 평균 평점 **3.0**

사용자 A의 어벤저스 3편 베이스 라인 평점 = $3.5 - 0.5 + 0.7 = 3.7$

모든 사용자의 평균 영화 평점 3.5	+	사용자 편향 점수 $3.0 - 3.5 = -0.5$	+	아이템 편향 점수 $4.2 - 3.5 = 0.7$
---------------------------------------	---	-------------------------------------	---	------------------------------------

어벤저스 3편 평균 평점
4.2



교차 검증과 하이퍼 파라미터 튜닝

- cross_validate()와 GridSearchCV 클래스 제공함
- cross_validate()는 surprise.model_selection 모듈 내에 존재함
- cross_validate()의 인자
 - 알고리즘 객체, 데이터, 성능 평가 방법, 폴드 데이터 세트 개수

```
from surprise.model_selection import cross_validate

# 판다스 DataFrame에서 Surprise 데이터 세트로 데이터 로딩
ratings = pd.read_csv('ratings.csv') # reading data in pandas df
reader = Reader(rating_scale=(0.5, 5.0))
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)

algo = SVD(random_state=0)
cross_validate(algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.8827	0.8646	0.8758	0.8764	0.8749	0.8749	0.0058
MAE (testset)	0.6768	0.6644	0.6738	0.6724	0.6709	0.6717	0.0041
Fit time	1.56	1.60	1.61	1.65	1.72	1.63	0.06
Test time	0.09	0.32	0.11	0.31	0.10	0.19	0.11

```
{'test_rmse': array([0.88265688, 0.86459918, 0.87581833, 0.87639267, 0.87486001]),
 'test_mae': array([0.67678983, 0.66439488, 0.67382928, 0.67240378, 0.6708778 ]),
 'fit_time': (1.5565800666809082,
 1.6041619777679443,
 1.6122362613677979,
 1.6494355201721191,
 1.7216756343841553),
 'test_time': (0.09142255783081055,
 0.32407689094543457,
 0.11034798622131348,
 0.3142893314361572,
 0.10209774971008301)}
```

- GridSearchCV 교차 검증 통한 하이퍼 파라미터 최적화 수행
- 하이퍼 파라미터 최적화는 알고리즘 유형에 따라 다를 수 있음

```

from surprise.model_selection import GridSearchCV

# 최적화할 파라미터를 딕셔너리 형태로 지정.
param_grid = {'n_epochs': [20, 40, 60], 'n_factors': [50, 100, 200] }

# CV를 3개 폴드 세트로 지정, 성능 평가는 rmse, mse로 수행하도록 GridSearchCV 구성
gs = GridSearchCV(SVD, param_grid, measures=['rmse', 'mae'], cv=3)
gs.fit(data)

# 최고 RMSE Evaluation 점수와 그때의 하이퍼 파라미터
print(gs.best_score['rmse'])
print(gs.best_params['rmse'])

0.8772390973772396
{'n_epochs': 20, 'n_factors': 50}

```

Surprise를 이용한 개인화 영화 추천 시스템 구축

- Surprise의 train_test_split() 분리 안 하면 오류남
- DatasetAutoFolds 클래스 사용하면 됨
- build_full_trainset() 메서드를 호출하면 전체 데이터를 학습 데이터 세트로 만들 수 있음

```

# 다음 코드는 train_test_split()으로 분리되지 않는 데이터 세트에 fit()을 호출해 오류가 남
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)
algo = SVD(n_factors=50, random_state=0)
algo.fit(data)

```

```

from surprise.dataset import DatasetAutoFolds

reader = Reader(line_format='user item rating timestamp', sep=',', rating_scale=(0.5, 5))
# DatasetAutoFolds 클래스를 ratings_noh.csv 파일 기반으로 생성.
data_folds = DatasetAutoFolds(ratings_file='ratings_noh.csv', reader=reader)

#전체 데이터를 학습데이터로 생성함.
trainset = data_folds.build_full_trainset()

```

```

algo = SVD(n_epochs=20, n_factors=50, random_state=0)
algo.fit(trainset)

```

```

<surprise.prediction_algorithms.matrix_factorization.SVD at 0x7affcab8d550>

```

- 특정 사용자는 userId = 9 인 사용자
- 평점 매기지 않은 movieId 42로 선정 후 예측 평점 계산

```
# 영화에 대한 상세 속성 정보 DataFrame로딩
movies = pd.read_csv('movies.csv')

# userId=9 의 movieId 데이터 추출하여 movieId=42 데이터가 있는지 확인.
movieIds = ratings[ratings['userId']==9]['movieId']
if movieIds[movieIds==42].count() == 0:
    print('사용자 아이디 9는 영화 아이디 42의 평점 없음')

print(movies[movies['movieId']==42])
```

userId	movieId	title	genres
38	42	Dead Presidents (1995)	Action Crime Drama

- movieId 42인 영화에 대해 userId 9 사용자의 추천 예상 평점은 predict() 메서드 이용하면 됨
- 학습된 SVD 객체에서 predict() 메서드 내에 user/movie Id 입력하면 됨
 - 모두 문자열이어야 함
- 추천 대상이 되는 영화를 추출

```

uid = str(9)
iid = str(42)

pred = algo.predict(uid, iid, verbose=True)

def get_unseen_surprise(ratings, movies, userId):
    #입력값으로 들어온 userId에 해당하는 사용자가 평점을 매긴 모든 영화를 리스트로 생성
    seen_movies = ratings[ratings['userId']== userId]['movieId'].tolist()

    # 모든 영화들의 movieId를 리스트로 생성.
    total_movies = movies['movieId'].tolist()

    # 모든 영화들의 movieId중 이미 평점을 매긴 영화의 movieId를 제외하여 리스트로 생성
    unseen_movies= [movie for movie in total_movies if movie not in seen_movies]
    print('평점 매긴 영화수:',len(seen_movies), '추천대상 영화수:',len(unseen_movies),
          '전체 영화수:',len(total_movies))

    return unseen_movies

unseen_movies = get_unseen_surprise(ratings, movies, 9)

```

평점 매긴 영화수: 46 추천대상 영화수: 9696 전체 영화수: 9742

- `recomm_movie_by_surprise()` 함수를 새롭게 생성
- 인자로 학습 완료된 추천 알고리즘 객체, 추천 대상 사용자 아이디, 추천 대상 영화 리스트 객체, 추천 상위 N개 개수를 받음
- 알고리즘 객체의 `predict()` 메서드 호출
- 결과인 `Prediction` 객체를 리스트 객체로 저장
- 객체를 다시 예측 평점 높은 순으로 정렬
- `Top_N`개의 `Prediction` 객체에서 영화 Id, 제목, 예측 평점 정보 추출하여 반환

```

def recomm_movie_by_surprise(algo, userID, unseen_movies, top_n=10):
    # 알고리즘 객체의 predict() 메서드를 평점이 없는 영화에 반복 수행한 후 결과를 list 객체
    predictions = [algo.predict(str(userID), str(movieId)) for movieId in unseen_movies]

    # predictions list 객체는 surprise의 Predictions 객체를 원소로 가지고 있음.
    # [Prediction(uid='9', iid='1', est=3.69), Prediction(uid='9', iid='2', est=2.98),...]
    # 이를 est 값으로 정렬하기 위해서 아래의 sortkey_est 함수를 정의함.
    # sortkey_est 함수는 list 객체의 sort() 함수의 키 값으로 사용되어 정렬 수행.
    def sortkey_est(pred):
        return pred.est

    # sortkey_est( ) 반환값의 내림 차순으로 정렬 수행하고 top_n개의 최상위 값 추출.
    predictions.sort(key=sortkey_est, reverse=True)
    top_predictions= predictions[:top_n]

    # top_n으로 추출된 영화의 정보 추출. 영화 아이디, 추천 예상 평점, 제목 추출
    top_movie_ids = [ int(pred.iid) for pred in top_predictions]
    top_movie_rating = [ pred.est for pred in top_predictions]
    top_movie_titles = movies[movies.movieId.isin(top_movie_ids)]['title']
    top_movie_preds = [ (id, title, rating) for id, title, rating in zip(top_movie_ids, top_movie_rating, top_movie_titles)]

    return top_movie_preds

unseen_movies = get_unseen_surprise(ratings, movies, 9)
top_movie_preds = recomm_movie_by_surprise(algo, 9, unseen_movies, top_n=10)
print('##### Top-10 추천 영화 리스트 #####')

for top_movie in top_movie_preds:
    print(top_movie[1], ":", top_movie[2])

```